

Exercícios — Aula 02

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

Todos os exercícios usam a mesma base de dados. Crie um arquivo `dados.js` com:

```
export const alunos = [
  { id: 1, nome: "Ana Souza", idade: 22, curso: "front-end", notas: [8, 7, 9], ativo: true },
  { id: 2, nome: "Bruno Lima", idade: 31, curso: "back-end", notas: [6, 5, 7], ativo: true },
  { id: 3, nome: "Carla Dias", idade: 25, curso: "front-end", notas: [10, 9, 9], ativo: false },
  { id: 4, nome: "Diego Alves", idade: 19, curso: "back-end", notas: [4, 6, 5], ativo: true },
  { id: 5, nome: "Elisa Martins", idade: 28, curso: "dados", notas: [9, 8, 10], ativo: true },
  { id: 6, nome: "Felipe Rocha", idade: 35, curso: "back-end", notas: [7, 7, 8], ativo: false },
];
```

Regra de ouro desta lista: nenhum exercício pode alterar o array `alunos`. Se ao final do seu script um `console.log(alunos)` mostrar algo diferente do original, você mutou sem querer.

Nível 1 — obrigatório

Exercício 1 — Aquecendo com `map` e `filter`

Em `01-basico.js`, imprima:

1. Um array só com os **nomes** dos alunos.
2. Um array só com os alunos **ativos**.
3. Um array de strings no formato `"Ana Souza (22 anos)"`.
4. Os alunos do curso de **back-end**.
5. Os nomes dos alunos com **mais de 25 anos**, em MAIÚSCULAS.

Exercício 2 — `reduce` na prática

Em `02-reduce.js`, calcule e imprima:

1. A **soma das idades** de todos os alunos.
2. A **média de idade** da turma (com uma casa decimal).
3. A **quantidade** de alunos ativos — usando `reduce`, não `filter().length`.
4. O aluno **mais velho** (o objeto inteiro).

Exercício 3 — Encadeando

Em `03-encadeando.js`, responda em **uma única expressão encadeada** cada item:

1. A média de idade **apenas dos alunos ativos**.
2. Os nomes dos alunos de front-end, em ordem alfabética.
3. Quantos alunos de back-end estão ativos.

Nível 2 — esperado

Exercício 4 — Boletim

Em `04-boletim.js`, crie as funções abaixo (todas puras: recebem dados, devolvem dados, **não** usam `console.log` por dentro):

Função	Devolve
<code>calcularMedia(notas)</code>	a média de um array de notas
<code>situacao(media)</code>	"Aprovado" (≥ 7), "Recuperação" (≥ 5) ou "Reprovado"
<code>gerarBoletim(alunos)</code>	novo array com <code>{ nome, media, situacao }</code> para cada aluno
<code>aprovados(alunos)</code>	só os aprovados, ordenados da maior média para a menor

Depois imprima um boletim formatado assim:

```
BOLETIM DA TURMA
  Elisa Martins    9.0  Aprovado
  Carla Dias      9.3  Aprovado
  ...

Aprovados: 4 de 6 (66.7%)
```

Exercício 5 — Imutabilidade

Em `05-imutavel.js`, escreva funções que **nunca alteram** o array recebido:

Função	O que faz
<code>adicionarAluno(alunos, novoAluno)</code>	devolve um novo array com o aluno no final
<code>removerAluno(alunos, id)</code>	devolve um novo array sem o aluno daquele id
<code>atualizarAluno(alunos, id, mudancas)</code>	devolve novo array com aquele aluno alterado
<code>desativarTodos(alunos)</code>	devolve novo array com todos os <code>ativo</code> em <code>false</code>

Ao final, imprima o array original e prove que ele continua exatamente como começou.

Dica para `atualizarAluno`: use `map` com um ternário e spread —
`aluno.id === id ? { ...aluno, ...mudancas } : aluno`.

Desafio — opcional

Exercício 6 — Relatório completo

Em `06-relatorio.js`, gere um relatório com:

1. **Alunos por curso** — um objeto no formato `{ "front-end": 2, "back-end": 3, "dados": 1 }`.
Dica: `reduce` com um objeto como acumulador.
2. **Média de notas por curso.**
3. **Ranking geral** de alunos por média, com posição (1º, 2º, 3º...).
4. **Estatísticas:** maior média, menor média, média geral da turma.
5. Uma tabela final formatada e alinhada no terminal (dica: `.padEnd(20)`).

Exercício 7 — Agrupamento genérico

Escreva uma função `agruparPor(lista, chave)` que funcione com **qualquer** array de objetos:

```
agruparPor(alunos, "curso");  
  
// {  
//   "front-end": [ {...Ana}, {...Carla} ],  
//   "back-end":  [ {...Bruno}, {...Diego}, {...Felipe} ],  
//   "dados":     [ {...Elisa} ]  
// }
```

Teste também com `agruparPor(alunos, "ativo")`. Se funcionar com os dois sem mudar nada na função, você entendeu o exercício.

Checklist de entrega

- ☐ Nenhum exercício alterou o array `alunos` original
- ☐ Usei `const` em tudo, exceto onde precisei reatribuir
- ☐ Usei arrow functions nos callbacks de `map` / `filter` / `reduce`
- ☐ Todo `reduce` tem valor inicial
- ☐ Minhas funções recebem dados e devolvem dados (o `console.log` fica só no final do arquivo)

